

Vehicle Service Monitoring System

ASP.NET Core MVC Laboratory Exercise Specification

Objective

Develop an ASP.NET Core MVC application that allows service advisors to register, log in, and monitor customer vehicles from check-in until release. Use Model Binding, DTOs, Data Annotation Validation, Cookie Authentication, Repository Pattern (in-memory), Razor Views, and Bootstrap.

Business Scenario

A small automotive service center wants to replace paper records with a simple web application. Reception staff register customer vehicles, monitor active service jobs, and record vehicle release times. Only authenticated users may access the system.

Functional Requirements

User Registration (First Name, Last Name, Email, Username, Password, Confirm Password); Login; Logout; Register Vehicle; View Service Monitoring List; Edit Vehicle Information; View Details; Release Vehicle; Search by Service Number, Customer Name, Plate Number, or Vehicle.

Vehicle Registration Fields

Service Number (auto-generated), Customer Name, Contact Number, Vehicle Make, Vehicle Model, Model Year, Plate Number, Vehicle Color, Service Type, Service Bay, Check-in Date & Time, Expected Release Date, Remarks.

Service Monitoring

Display Service Number, Customer, Vehicle, Plate Number, Service Type, Service Bay, Check-in Time, Release Time, Status, Actions. Release Time remains blank until released.

Status

Waiting, In Service, Ready for Release, Released.

Validation

Use Required, StringLength, Range, Compare, EmailAddress, Phone, Display and DataType attributes. Validate ModelState before saving.

Authentication

Public: Login, Register. Protected: Vehicle pages. Implement Cookie Authentication and [Authorize].

Entities

User: Id, FirstName, LastName, Email, Username, Password. ServiceJob: Id, ServiceNumber, CustomerName, ContactNumber, VehicleMake, VehicleModel, ModelYear, PlateNumber, VehicleColor, ServiceType, ServiceBay, CheckInDateTime, ExpectedReleaseDate, ActualReleaseDateTime, Status, Remarks.

DTO Guidelines

Students decide appropriate DTOs such as RegistrationDto, LoginDto, ServiceJobCreateDto, ServiceJobEditDto, ServiceJobReleaseDto, and DetailsDto.

Repositories

Implement UserRepository and ServiceJobRepository using static List. Controllers must not store application data directly. No database is required.

Controllers

HomeController, AccountController, ServiceJobController.

Views

Login, Register, Index (Monitoring), Create, Edit, Details, Release.

End-to-End User Story

Register an account, log in, register a vehicle, monitor active jobs, edit service information, release the vehicle, and verify that the monitoring page shows both check-in and release times.

Submission Requirements

Submit a complete ASP.NET Core MVC solution with Bootstrap UI, Cookie Authentication, Model Binding, DTOs, Repository Pattern, CRUD operations, search, validation, protected pages, and clean code organization.

Grading Rubric (100 pts)

Authentication-15, CRUD-25, Validation-15, Model Binding/DTOs-10, Repository Pattern-10, Search-10, UI/Bootstrap-10, Code Quality-5.